KM+ ECOMMERCE · QA

Test Results — KM+ Ecommerce Platform

Document version: 1.0
Test date: 2026-06-30
Environment: Node.js 22.22.2, Ubuntu 22.04
Run command: `node tests/run_unit_tests.js`

1. Unit Test Suite

The unit tests cover the three pure-logic backend modules that implement core business rules: pricing/discount computation, cart quantity enforcement, and referral code logic. These modules have no database or HTTP dependencies and can be exercised directly.

Command:

```
node tests/run_unit_tests.js
```

Full output (captured 2026-06-30T07:14:39Z):

```
PASS | computeMileageDiscount: 0km = no discount                                | 0ms
PASS | computeMileageDiscount: 4999km = no discount (below tier)                | 1ms
PASS | computeMileageDiscount: 5000km = RM2 (200 sen)                           | 0ms
PASS | computeMileageDiscount: 19999km = RM2                                     | 0ms
PASS | computeMileageDiscount: 20000km = RM5 (500 sen)                           | 0ms
PASS | computeMileageDiscount: 49999km = RM5                                     | 0ms
PASS | computeMileageDiscount: 50000km = RM10 (1000 sen)                         | 0ms
PASS | computeMileageDiscount: 100000km = RM10 (top tier capped)                 | 0ms
PASS | computeMileageDiscount: null = no discount                                | 0ms
PASS | computeMileageDiscount: negative = no discount                            | 0ms
PASS | computeSubtotal: empty array = 0                                          | 0ms
PASS | computeSubtotal: single line                                              | 0ms
PASS | computeSubtotal: multiple lines                                           | 0ms
PASS | computeSubtotal: non-array input = 0                                      | 0ms
PASS | computeBaseDeliveryFee: collection always free                            | 0ms
PASS | computeBaseDeliveryFee: delivery below threshold = RM15 base              | 0ms
PASS | computeBaseDeliveryFee: delivery exactly at RM200 = free                  | 0ms
PASS | computeBaseDeliveryFee: delivery above threshold = free                   | 0ms
PASS | computeOrderTotals: collection — no delivery, discount not applied       | 0ms
PASS | computeOrderTotals: delivery with RM5 mileage discount                   | 0ms
PASS | computeOrderTotals: RM10 discount on RM15 fee → RM5 delivery remaining | 0ms
PASS | computeOrderTotals: free-delivery threshold — mileage discount = 0      | 0ms
PASS | computeOrderTotals: no motorcycle profile = no discount                   | 0ms
PASS | formatCentsAsRM: 19900 → RM 199.00                                        | 0ms
PASS | formatCentsAsRM: 0 → RM 0.00                                              | 0ms
PASS | computeAddToCart: new item, stock available                               | 1ms
```

```
PASS | computeAddToCart: existing item — quantities summed                    | 0ms
PASS | computeAddToCart: exceeds available stock                               | 0ms
PASS | computeAddToCart: exceeds MAX_QUANTITY_PER_ITEM cap                     | 0ms
PASS | computeAddToCart: zero quantity rejected                                | 0ms
PASS | validateQuantityUpdate: valid integer within stock                      | 0ms
PASS | validateQuantityUpdate: zero rejected                                   | 0ms
PASS | validateQuantityUpdate: float rejected                                  | 0ms
PASS | validateQuantityUpdate: exceeds stock rejected                          | 0ms
PASS | validateQuantityUpdate: exceeds cap (20) rejected                       | 0ms
PASS | partitionCartByAvailability: 1 valid, 2 invalid                         | 0ms
PASS | generateReferralCode: 8 chars                                           | 0ms
PASS | generateReferralCode: no ambiguous chars (0,O,1,I,L)                    | 0ms
PASS | normaliseReferralCode: uppercase and strip non-alphanumeric             | 0ms
PASS | normaliseReferralCode: null → empty string                              | 0ms
PASS | isValidCodeFormat: valid 8-char code                                    | 0ms
PASS | isValidCodeFormat: too short rejected                                   | 0ms
PASS | isValidCodeFormat: ambiguous char "O" rejected                          | 0ms
PASS | validateReferralCodeUsage: no code = valid (referral optional)          | 1ms
PASS | validateReferralCodeUsage: code not in DB = invalid                     | 0ms
PASS | validateReferralCodeUsage: inactive code = invalid                      | 0ms
PASS | validateReferralCodeUsage: active code = valid                          | 0ms
PASS | validateReferralCodeUsage: self-referral blocked                        | 0ms
PASS | computeCommission: 5% of RM100 = 500 sen                                | 0ms
PASS | computeCommission: floors fractional cents                                | 0ms
PASS | computeCommission: zero total = 0                                       | 0ms
PASS | computeCommission: zero pct = 0                                         | 0ms

Results: 52 passed, 0 failed, 52 total | 21ms
```

Exit code: 0 ✓

---

2. Test Case Reference

lib/pricing.js — Mileage Discount Tiers (10 tests)

The mileage discount is a 3-tier system rewarding high-mileage riders with a reduction on their delivery fee. Tiers are checked in descending mileage order (highest tier first).

Tier	Mileage Range	Discount
Top	≥ 50,000 km	RM 10.00 (1000 sen)
Mid	20,000 – 49,999 km	RM 5.00 (500 sen)
Entry	5,000 – 19,999 km	RM 2.00 (200 sen)
None	< 5,000 km or null	—

#	Test	Input	Expected	Result
1	Below any tier	`0` km	`0` sen	PASS
2	Just below entry tier	`4999` km	`0` sen	PASS

#	Test	Input	Expected	Result
3	Entry tier boundary	`5000` km	`200` sen	PASS
4	Mid-entry tier	`19999` km	`200` sen	PASS
5	Mid tier boundary	`20000` km	`500` sen	PASS
6	Just below top tier	`49999` km	`500` sen	PASS
7	Top tier boundary	`50000` km	`1000` sen	PASS
8	Top tier (no higher tier)	`100000` km	`1000` sen	PASS
9	Null (no motorcycle)	`null`	`0` sen	PASS
10	Negative value	`-100`	`0` sen	PASS

lib/pricing.js — Subtotal (4 tests)

#	Test	Input	Expected	Result
11	Empty cart	`[]`	`0`	PASS
12	Single line	`[{price:5000, qty:2}]`	`10000`	PASS
13	Multiple lines	`[{5000,2}, {1500,3}]`	`14500`	PASS
14	Non-array	`null`	`0` (safe)	PASS

lib/pricing.js — Delivery Fee (4 tests)

Free-delivery threshold: RM 200.00 (20000 sen). Base fee: RM 15.00 (1500 sen).

#	Test	Method	Subtotal	Expected	Result
15	Collection always free	`collection`	any	`0`	PASS
16	Below threshold	`delivery`	RM 50	`1500` sen	PASS
17	Exactly at threshold	`delivery`	RM 200	`0`	PASS
18	Above threshold	`delivery`	RM 300	`0`	PASS

lib/pricing.js — Full Order Totals (5 tests)

#	Test	Scenario	Result
19	Collection + 60k km rider	No delivery fee, mileage discount not applied (nothing to discount)	PASS
20	Delivery + 25k km	`mileageDiscountCents=500`, `deliveryFeeCents=1000`	PASS
21	Discount cap	50k km → RM10 discount on RM15 fee → RM5 delivery (never negative)	PASS

#	Test	Scenario	Result
22	Free threshold + 50k km	Base already free → discount = 0	PASS
23	No motorcycle	`mileageKm=null` → discount = 0, full delivery fee	PASS

lib/pricing.js — Formatting (2 tests)

#	Test	Input	Expected	Result
24	Standard amount	`19900`	`'RM 199.00'`	PASS
25	Zero	`0`	`'RM 0.00'`	PASS

lib/cart.js — Add to Cart (5 tests)

#	Test	Input	Expected	Result
26	New item	qty=2, stock=10	`ok=true, newQuantity=2`	PASS
27	Existing item	current=3, add=2, stock=10	`ok=true, newQuantity=5`	PASS
28	Exceeds stock	current=3, add=10, stock=5	`ok=false`	PASS
29	Exceeds cap (20)	add=25, stock=100	`ok=false`	PASS
30	Zero quantity	add=0	`ok=false`	PASS

lib/cart.js — Quantity Update (5 tests)

#	Test	Input	Expected	Result
31	Valid update	qty=5, stock=10	`ok=true`	PASS
32	Zero	qty=0	`ok=false`	PASS
33	Float	qty=2.5	`ok=false`	PASS
34	Exceeds stock	qty=15, stock=10	`ok=false`	PASS
35	Exceeds cap	qty=25, stock=100	`ok=false`	PASS

lib/cart.js — Availability Partition (1 test)

#	Test	Setup	Expected	Result
36	Mixed cart	v1 valid; v2 qty>stock; v3 deactivated	`valid=[v1]`, `invalid=[v2,v3]`	PASS

lib/referral.js — Code Generation (2 tests)

#	Test	Expected	Result
37	8-character code	length = 8	PASS
38	No ambiguous chars	regex `/[0O1IL]/` does not match	PASS

lib/referral.js — Normalisation & Format (5 tests)

#	Test	Input	Expected	Result
39	Strip/uppercase	`'code: ab-cd-1234'`	`'CODEABCD1234'`	PASS
40	Null input	`null`	`''`	PASS
41	Valid 8-char	`'ABCD2345'`	`true`	PASS
42	Too short	`'ABC'`	`false`	PASS
43	Ambiguous char	`'ABCD234O'` (letter O)	`false`	PASS

lib/referral.js — Usage Validation (5 tests)

#	Test	Scenario	Expected	Result
44	No code entered	code= `''`	`valid=true` (referral is optional)	PASS
45	Code not in DB	record= `null`	`valid=false`	PASS
46	Inactive code	`isActive=false`	`valid=false`	PASS
47	Active code	`isActive=true`	`valid=true`	PASS
48	Self-referral	entered = own code	`valid=false`	PASS

lib/referral.js — Commission Calculation (4 tests)

#	Test	Input	Expected	Result
49	Standard 5%	RM100 order	`500` sen	PASS
50	Floors fractions	RM100.01 order	`500` sen (floor, not round)	PASS
51	Zero order total	`0`	`0`	PASS
52	Zero commission pct	`0%`	`0`	PASS

3. Demo Materials

Static UI mockups are in `demo/screenshots/`. No live demo is included.

File	What it shows
`01_product_listing.svg`	Product catalog grid — brand/category pills, variant price display, out-of-stock badge, quick-add button
`02_cart.svg`	Cart page — line items with optimistic quantity stepper, remove action, order summary sidebar
`03_checkout.svg`	Checkout — courier vs collection toggle, retail partner selection, mileage discount banner, totals
`04_payment_success.svg`	PaymentCallback success state — confirmed order number, paid status, view-order CTA
`05_admin_dashboard.svg`	Admin orders view — stats cards, filterable order table, status badges
`06_account_affiliate.svg`	Account affiliate tab — referral code display, payout bank details, referral history table

Real request/response payloads for all major endpoints are documented in `demo/api_examples.md`.

---

4. Summary

Module	Tests	Pass	Fail
`lib/pricing.js`	25	25	0
`lib/cart.js`	16	16	0
`lib/referral.js`	16	16	0
Total	**52**	**52**	**0**

All 52 tests run against the actual production modules — no mocks. Runtime: 21ms.

---

5. Notable Test Behaviours

Mileage discount cap (TC-21): The mileage discount is capped at `baseDeliveryFeeCents` — it can never produce a negative delivery fee. A 50,000km rider with a RM10 discount on a RM15 delivery fee pays RM5 (not RM0 or negative). This is implemented as `Math.min(discount, baseDeliveryFee)`.

Free-threshold interaction with mileage discount (TC-22): When the subtotal exceeds the free-delivery threshold (RM200), `baseDeliveryFeeCents = 0`. The mileage discount is then `Math.min(1000, 0) = 0` — correct, since there is nothing to discount. This means a high-mileage rider with a large order neither gets extra discount nor a negative fee.

Collection disables mileage discount (TC-19): Self-collection orders have zero delivery cost. The system explicitly sets `mileageDiscountCents = 0` for collection orders, avoiding a confusing UI display of "RM10 discount applied" on something that was already free.

Integer cents throughout: All money is stored and computed as integer sen (cents). `computeCommission` uses `Math.floor` rather than rounding (TC-50) — partial sen is always truncated in the affiliate's disfavour, preventing accumulated float rounding errors from overclaiming commission.

Referral ambiguous-character exclusion (TC-38): The charset `CODE_CHARSET` deliberately excludes `0, O, 1, I, L` to prevent transcription errors when referral codes are shared verbally or handwritten. This is verified by asserting that a generated code does not match the regex `/[0O1IL]/`.